

## 1. What is Bash?

Bash is a **command-line shell and scripting language** used mainly in Linux.

### Syntax:

```
#!/bin/bash
```

Example:

```
#!/bin/bash
```

```
echo "Hello World"
```

## 2. Running a Bash Script

### Purpose:

To execute a collection of Linux commands automatically.

Create a script

```
nano hello.sh
```

Write

```
#!/bin/bash
```

```
echo "Hello Asif"
```

Give execute permission

```
chmod +x hello.sh
```

Run

```
./hello.sh
```

## 3. Variables & Data Types

### 3.1 Introduction to Variables

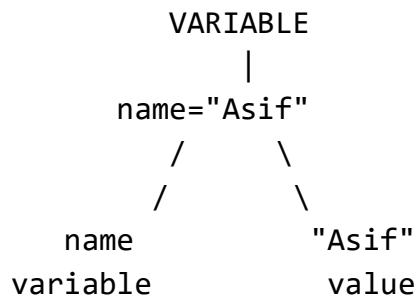
A variable is a named container used to store a value.

name="Asif"

Here:

- name → variable name
- "Asif" → value
- = → assignment operator

| Symbol | Purpose            | Example             |
|--------|--------------------|---------------------|
| =      | Assign value       | name="Asif"         |
| \$     | Get variable value | echo "\$name"       |
| " "    | Double quotes      | "Hello \$name"      |
| ' '    | Literal text       | '\$name'            |
| \${ }  | Variable expansion | "\${name}_DevOps"   |
| #      | Comment            | # This is a comment |



Local variable:

A **local variable** is a variable whose value is available only within the **current shell or function**, depending on where it is created.

| Symbol / Keyword | Purpose                                        | Example             |
|------------------|------------------------------------------------|---------------------|
| local            | Creates a variable only inside a function      | local name="Asif"   |
| =                | Assigns a value to the variable                | local name="Asif"   |
| \$               | Gets the variable value                        | echo "\$name"       |
| { }              | Defines the function block                     | check() { ... }     |
| \$1              | Gets the first argument passed to the function | local service="\$1" |

## Environmental Variables:

An environment variable is a variable that is available to the current shell and can be passed to programs/processes started from that shell.

| Symbol / Keyword | Purpose                                       | Example                        |
|------------------|-----------------------------------------------|--------------------------------|
| export           | Creates an environment variable               | export<br>APP_ENV="production" |
| =                | Assigns a value to the variable               | APP_ENV="production"           |
| \$               | Gets the variable value                       | echo "\$APP_ENV"               |
| \${}             | Expands the variable value                    | echo<br>"\${APP_ENV}_server"   |
| printenv         | Displays the value of an environment variable | printenv APP_ENV               |
| env              | Displays environment variables                | env                            |
| unset            | Removes an environment variable               | unset APP_ENV                  |

## Simple Example

```
#!/bin/bash
```

```
export APP_ENV="production"
```

```
echo "$APP_ENV"
```

```
printenv APP_ENV
```

Output:

```
production  
production
```

## Positional Parameters (Script Arguments):

Positional parameters are special variables used to receive values/arguments given to a Bash script when the script is executed.

| Symbol / Keyword | Purpose                                                 | Example    |
|------------------|---------------------------------------------------------|------------|
| \$0              | Gets the name of the script                             | echo "\$0" |
| \$1              | Gets the first argument passed to the script            | echo "\$1" |
| \$2              | Gets the second argument passed to the script           | echo "\$2" |
| \$3              | Gets the third argument passed to the script            | echo "\$3" |
| \$#              | Gets the total number of arguments passed to the script | echo "\$#" |
| \$@              | Gets all arguments passed to the script                 | echo "\$@" |

Special variable:

Special variables are predefined Bash variables that provide information about the script, its arguments, processes, and the result of commands.

| Symbol / Keyword | Purpose                                                      | Example     |
|------------------|--------------------------------------------------------------|-------------|
| \$0              | Gets the name of the script                                  | echo "\$0"  |
| \$1, \$2, \$3    | Gets script arguments                                        | echo "\$1"  |
| \$#              | Gets the number of arguments                                 | echo "\$#"  |
| \$@              | Gets all arguments separately                                | echo "\$@"  |
| \$*              | Gets all arguments as one value                              | echo "\$*"  |
| \$?              | Gets the exit status of the last command                     | echo "\$?"  |
| \$\$             | Gets the process ID (PID) of the current script              | echo "\$\$" |
| \$_              | Gets the PID of the most recently started background process | echo "\$_"  |

### Read-Only Variables:

A read-only variable is a variable whose value cannot be changed or reassigned after it is created.

| Symbol / Keyword | Purpose                                                                   | Example                 |
|------------------|---------------------------------------------------------------------------|-------------------------|
| readonly         | Creates a read-only variable                                              | readonly<br>NAME="Asif" |
| =                | Assigns the initial value                                                 | readonly<br>PORT=8080   |
| \$               | Gets the value of the variable                                            | echo "\$PORT"           |
| unset            | Attempts to remove the variable, but a read-only variable cannot be unset | unset PORT              |

### Unset Variables:

An unset variable is a variable that has been removed or has not been assigned a value.

| Symbol / Keyword | Purpose                          | Example            |
|------------------|----------------------------------|--------------------|
| unset            | Removes a variable               | unset NAME         |
| =                | Assigns a value to a variable    | NAME="Asif"        |
| \$               | Gets the variable value          | echo "\$NAME"      |
| -v               | Checks whether a variable is set | [[ -v<br>NAME ]]   |
| \${}             | Expands the variable value       | echo<br>"\${NAME}" |

### Array Variables:

An array variable is a variable that can store multiple values under one variable name.

| Symbol / Keyword | Purpose                                     | Example                             |
|------------------|---------------------------------------------|-------------------------------------|
| ()               | Creates an array and stores multiple values | servers=("nginx" "apache" "tomcat") |
| =                | Assigns values to the array                 | servers=("nginx" "apache")          |
| [@]              | Gets all elements of the array              | echo "\${servers[@]}"               |
| [0]              | Gets the first element of the array         | echo "\${servers[0]}"               |
| [1]              | Gets the second element of the array        | echo "\${servers[1]}"               |
| \${#array[@]}    | Gets the number of elements in the array    | echo "\${#servers[@]}"              |
| unset            | Removes an array or an array element        | unset 'servers[1]'                  |

Integer Variables:

An integer variable is a variable used to store whole numbers and perform arithmetic calculations in Bash.

| Symbol / Keyword | Purpose                         | Example            |
|------------------|---------------------------------|--------------------|
| declare -i       | Creates an integer variable     | declare -i age=25  |
| =                | Assigns a value                 | age=25             |
| \$               | Gets the variable value         | echo "\$age"       |
| +                | Addition                        | sum=\$((a + b))    |
| -                | Subtraction                     | result=\$((a - b)) |
| *                | Multiplication                  | result=\$((a * b)) |
| /                | Division                        | result=\$((a / b)) |
| %                | Gets the remainder              | result=\$((a % b)) |
| \$(( ))          | Performs arithmetic calculation | sum=\$((10 + 20))  |

Best Practices

Real DevOps Use Case Example:

Best practices are recommended methods for writing Bash variables in a clean, safe, readable, and reusable way.

| Best Practice                              | Purpose                                              | Example                                    |
|--------------------------------------------|------------------------------------------------------|--------------------------------------------|
| Use meaningful names                       | Makes the script easy to understand                  | <code>server_name="nginx"</code>           |
| Use lowercase for local variables          | Helps identify function/local variables              | <code>local service="\$1"</code>           |
| Use uppercase for configuration variables  | Makes important settings easy to identify            | <code>APP_PORT=8080</code>                 |
| Use <code>local</code> inside functions    | Prevents accidental changes to other variables       | <code>local status</code>                  |
| Use quotes                                 | Prevents problems with spaces and special characters | <code>echo "\$name"</code>                 |
| Use <code>readonly</code> for fixed values | Prevents accidental changes                          | <code>readonly<br/>APP_NAME="myapp"</code> |
| Use comments                               | Explains what the code does                          | <code># Check nginx<br/>status</code>      |
| Avoid unnecessary variables                | Keeps the script simple                              | <code>echo "\$APP_NAME"</code>             |

Operators:

Operators are symbols or keywords used to perform operations on values and variables.

Arithmetic Operators:

Arithmetic operators are symbols used to perform mathematical calculations such as addition, subtraction, multiplication, division, and remainder.

| Symbol / Keyword | Purpose                                         | Example           |
|------------------|-------------------------------------------------|-------------------|
| +                | Addition — adds two numbers                     | echo \$((10 + 5)) |
| -                | Subtraction — subtracts one number from another | echo \$((10 - 5)) |
| *                | Multiplication — multiplies two numbers         | echo \$((10 * 5)) |
| /                | Division — divides one number by another        | echo \$((10 / 5)) |
| %                | Modulus — gives the remainder after division    | echo \$((10 % 3)) |
| **               | Exponent — raises a number to a power           | echo \$((2 ** 3)) |
| ++               | Increment — increases a value by 1              | ((count++))       |
| --               | Decrement — decreases a value by 1              | ((count--))       |
| \$(( ))          | Performs arithmetic calculation                 | sum=\$((10 + 20)) |

#### Relational Operators:

Relational operators are used to compare two numbers and determine their relationship, such as equal, greater than, or less than.

| Symbol / Keyword | Purpose                                                           | Example           |
|------------------|-------------------------------------------------------------------|-------------------|
| -eq              | Checks if two numbers are equal                                   | [[ \$a -eq \$b ]] |
| -ne              | Checks if two numbers are not equal                               | [[ \$a -ne \$b ]] |
| -gt              | Checks if the first number is greater than the second             | [[ \$a -gt \$b ]] |
| -lt              | Checks if the first number is less than the second                | [[ \$a -lt \$b ]] |
| -ge              | Checks if the first number is greater than or equal to the second | [[ \$a -ge \$b ]] |

|     |                                                                |                                |
|-----|----------------------------------------------------------------|--------------------------------|
| -le | Checks if the first number is less than or equal to the second | <code>[[ \$a -le \$b ]]</code> |
|-----|----------------------------------------------------------------|--------------------------------|

Logical Operators:

Logical operators are used to combine two or more conditions and make decisions based on whether the conditions are true or false.

| Symbol / Keyword | Purpose                                   | Example                                            |
|------------------|-------------------------------------------|----------------------------------------------------|
| &&               | <b>AND</b> — both conditions must be true | <code>[[ \$a -gt 5 &amp;&amp; \$b -lt 10 ]]</code> |
| ,                |                                           | ,                                                  |
| !                | <b>NOT</b> — reverses the condition       | <code>[[ ! -f "file.txt" ]]</code>                 |

String Operators:

String operators are used to compare or check text (strings) in Bash.

| Symbol / Keyword | Purpose                                                  | Example                               |
|------------------|----------------------------------------------------------|---------------------------------------|
| ==               | Checks if two strings are equal                          | <code>[[ "\$name" == "Asif" ]]</code> |
| !=               | Checks if two strings are not equal                      | <code>[[ "\$name" != "Asif" ]]</code> |
| -z               | Checks if a string is empty                              | <code>[[ -z "\$name" ]]</code>        |
| -n               | Checks if a string is not empty                          | <code>[[ -n "\$name" ]]</code>        |
| <                | Checks if one string comes before another alphabetically | <code>[[ "\$a" &lt; "\$b" ]]</code>   |
| >                | Checks if one string comes after another alphabetically  | <code>[[ "\$a" &gt; "\$b" ]]</code>   |

File Test Operators:

File test operators are used to check the properties of files and directories, such as whether they exist, are readable, writable, or executable.

| Symbol / Keyword | Purpose                                         | Example              |
|------------------|-------------------------------------------------|----------------------|
| -e               | Checks whether a file or directory exists       | [[ -e "file.txt" ]]  |
| -f               | Checks whether it is a regular file             | [[ -f "file.txt" ]]  |
| -d               | Checks whether it is a directory                | [[ -d "backup" ]]    |
| -r               | Checks whether the file is readable             | [[ -r "file.txt" ]]  |
| -w               | Checks whether the file is writable             | [[ -w "file.txt" ]]  |
| -x               | Checks whether the file is executable           | [[ -x "script.sh" ]] |
| -s               | Checks whether the file exists and is not empty | [[ -s "file.txt" ]]  |
| -L               | Checks whether it is a symbolic link            | [[ -L "link.txt" ]]  |

Ternary Operator:

A ternary operator is a short way of writing an **if-else** decision.

| Symbol / Keyword | Purpose                                                                | Example                                            |
|------------------|------------------------------------------------------------------------|----------------------------------------------------|
| &&               | Executes the next command when the condition is true                   | [[ \$age -ge 18 ]] && echo "Adult"                 |
|                  | Executes the next command when the previous command/condition is false | [[ \$age -ge 18 ]] && echo "Adult"    echo "Minor" |
| [[ ]]            | Tests a condition                                                      | [[ \$age -ge 18 ]]                                 |
| ? :              | Traditional ternary syntax — <b>not directly supported by Bash</b>     | condition ? true : false                           |

Best Practices for Operators

Real DevOps Use Case Example:

Best practices for operators are recommended ways to use Bash operators correctly, safely, and clearly when writing scripts

| Best Practice                                                          | Purpose                                                       | Example                                    |
|------------------------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------|
| Use <code>[[ ]]</code> for conditions                                  | Provides safer and cleaner condition testing                  | <code>[[ \$age -gt 18 ]]</code>            |
| Use <code>-eq</code> , <code>-gt</code> , <code>-lt</code> for numbers | Correctly compares numeric values                             | <code>[[ \$a -gt \$b ]]</code>             |
| Use <code>==</code> , <code>!=</code> for strings                      | Correctly compares text values                                | <code>[[ "\$env" == "production" ]]</code> |
| Quote variables                                                        | Prevents problems with spaces and special characters          | <code>[[ "\$name" == "Asif" ]]</code>      |
| Use <code>&amp;&amp;</code> carefully                                  | Runs the next command only when the previous command succeeds | <code>command1 &amp;&amp; command2</code>  |
| Use <code>`</code>                                                     |                                                               | <code>`</code> for alternatives            |
| Use file operators before file operations                              | Prevents errors when files/directories are missing            | <code>[[ -f "\$file" ]]</code>             |
| Use parentheses for calculations                                       | Makes arithmetic operations clear                             | <code>total=\$((a + b))</code>             |
| Prefer <code>if-else</code> for complex logic                          | Makes scripts easier to read                                  | <code>if [[ ... ]]; then ... fi</code>     |
| Add spaces inside <code>[[ ]]</code>                                   | Required for correct Bash syntax                              | <code>[[ \$a -eq \$b ]]</code>             |

| Category                   | Purpose                 | Operators                                        |
|----------------------------|-------------------------|--------------------------------------------------|
| <b>Numbers</b>             | Compare numbers         | <code>-eq -ne -gt -lt -ge -le</code>             |
| <b>Strings</b>             | Compare or check text   | <code>== != -z -n</code>                         |
| <b>Files</b>               | Check files/directories | <code>-f -d -e -r -w -x</code>                   |
| <b>Multiple conditions</b> | Combine conditions      | <code>&amp;&amp;    !</code>                     |
| <b>Calculations</b>        | Perform arithmetic      | <code>+ - * / %</code> with <code>\$(( ))</code> |
| <b>Conditions</b>          | Test conditions safely  | Prefer <code>[[ ]]</code>                        |

## Introduction to Conditional Statements:

**Conditional statements are used to make decisions in a Bash script based on whether a condition is true or false.**

In simple words:

**“If this condition is true, do this; otherwise, do something else.”**

### Why do we use Conditional Statements?

In DevOps, scripts often need to **check something and then take an action**.

For example:

- Check whether a directory exists.
- Check whether Nginx is running.
- Check whether a file exists.
- Check whether disk usage is high.
- Check whether a service needs to be restarted.

| Statement / Keyword | Purpose                                                        | Example                     |
|---------------------|----------------------------------------------------------------|-----------------------------|
| if                  | Executes code when a condition is true                         | if [[ \$age -ge 18 ]]; then |
| then                | Starts the commands to execute when the condition is true      | then echo "Adult"           |
| else                | Executes code when the condition is false                      | else echo "Minor"           |
| elif                | Checks another condition if the previous condition is false    | elif [[ \$age -ge 13 ]]     |
| fi                  | Ends the if statement                                          | fi                          |
| case                | Checks a value against multiple possible choices               | case "\$choice" in          |
| [[ ]]               | Tests a condition safely                                       | [[ -f "\$file" ]]           |
| &&                  | Executes the next command when the previous condition succeeds | condition && echo "Yes"     |
|                     | Executes the next command when the previous condition fails    | condition    echo "No"      |

if Statement:

The **if** statement is used to execute a block of commands only when a specified condition is true.

In simple words:

**IF something is true → DO something.**

| Symbol / Keyword   | Purpose                                                       | Example                            |
|--------------------|---------------------------------------------------------------|------------------------------------|
| <b>if</b>          | Starts the conditional statement                              | <code>if [[ \$age -ge 18 ]]</code> |
| <code>[[ ]]</code> | Tests whether the condition is true or false                  | <code>[[ \$age -ge 18 ]]</code>    |
| <code>-ge</code>   | Checks whether one number is greater than or equal to another | <code>[[ \$age -ge 18 ]]</code>    |
| <b>then</b>        | Starts the commands to execute when the condition is true     | <code>then echo "Adult"</code>     |
| <b>fi</b>          | Ends the <b>if</b> statement                                  | <code>fi</code>                    |

Simple Example:

```
#!/bin/bash
```

```
age=20
```

```
if [[ $age -ge 18 ]]
```

```
then
```

```
    echo "You are an adult"
```

```
fi
```

## Output

You are an adult

## Easy Memory

if → Start condition  
then → Do this if TRUE  
fi → End condition

if-else Statement:

The if-else statement is used to choose between two actions: one action when the condition is true and another action when the condition is false.

| Symbol / Keyword | Purpose                                        | Example               |
|------------------|------------------------------------------------|-----------------------|
| if               | Starts the condition                           | if [[ \$age -ge 18 ]] |
| [[ ]]            | Tests the condition                            | [[ \$age -ge 18 ]]    |
| then             | Starts the commands when the condition is true | then echo "Adult"     |
| else             | Runs when the condition is false               | else echo "Minor"     |
| fi               | Ends the if-else statement                     | fi                    |

## Simple Example

```
#!/bin/bash
```

```
age=16
```

```
if [[ $age -ge 18 ]]
then
    echo "You are an adult"
else
    echo "You are a minor"
fi
```

## Output

You are a minor

## Easy Memory

if        → Check condition  
then     → Action if TRUE  
else     → Action if FALSE  
fi        → End

if-elif-else Statement:

The **if-elif-else** statement is used when we need to check multiple conditions and perform a different action for each condition.

| Symbol / Keyword | Purpose                                                       | Example                   |
|------------------|---------------------------------------------------------------|---------------------------|
| if               | Starts the first condition                                    | if [[ \$marks -ge 90 ]]   |
| [[ ]]            | Tests the condition                                           | [[ \$marks -ge 90 ]]      |
| then             | Starts the commands when the condition is true                | then echo "Excellent"     |
| elif             | Checks another condition when the previous condition is false | elif [[ \$marks -ge 60 ]] |
| else             | Runs when all conditions are false                            | else echo "Fail"          |
| fi               | Ends the if-elif-else statement                               | fi                        |

## Simple Example

```
#!/bin/bash
```

```
marks=75
```

```
if [[ $marks -ge 90 ]]
```

```

then
    echo "Excellent"
elif [[ $marks -ge 60 ]]
then
    echo "Good"
else
    echo "Fail"
fi

```

## Output

Good

## Easy Memory

```

if      → Check first condition
elif    → Check another condition
else    → If none of the conditions are true
fi      → End the statement

```

case Statement:

The `case` statement is used to compare one value against multiple possible choices and execute the commands for the matching choice.

| Symbol / Keyword  | Purpose                                 | Example                                         |
|-------------------|-----------------------------------------|-------------------------------------------------|
| <code>case</code> | Starts the <code>case</code> statement  | <code>case "\$choice"</code><br><code>in</code> |
| <code>in</code>   | Separates the value from the choices    | <code>case "\$choice"</code><br><code>in</code> |
| <code>)</code>    | Marks the end of a pattern/choice       | <code>1)</code>                                 |
| <code>;;</code>   | Ends the commands for a matching choice | <code>echo "Start";;</code>                     |
| <code>*</code>    | Default option when no pattern matches  | <code>*)</code>                                 |
| <code>esac</code> | Ends the <code>case</code> statement    | <code>esac</code>                               |

## Simple Example

```
#!/bin/bash

choice=2

case "$choice" in
    1)
        echo "Start"
        ;;
    2)
        echo "Stop"
        ;;
    3)
        echo "Restart"
        ;;
    *)
        echo "Invalid choice"
        ;;
esac
```

## Output

Stop

## ★ Easy Memory

```
case → Start checking choices
in   → Start the patterns
)    → Define a choice
;;   → End that choice
*    → Default choice
esac → End case
```

Test Command [ and [[:

The test command is used to check whether a condition is true or false .

Bash provides two common ways to test conditions:

- `[ ]` → traditional test syntax
- `[[ ]]` → modern Bash conditional syntax

Both are commonly used with `if`.

| Symbol / Keyword        | Purpose                                                                   | Example                                            |
|-------------------------|---------------------------------------------------------------------------|----------------------------------------------------|
| <code>[ ]</code>        | Tests a condition using traditional Bash test syntax                      | <code>[ "\$age" -ge 18 ]</code>                    |
| <code>[[ ]]</code>      | Tests a condition using Bash's safer and more powerful conditional syntax | <code>[[ \$age -ge 18 ]]</code>                    |
| <code>-f</code>         | Checks whether a regular file exists                                      | <code>[[ -f "test.txt" ]]</code>                   |
| <code>-d</code>         | Checks whether a directory exists                                         | <code>[[ -d "backup" ]]</code>                     |
| <code>-eq</code>        | Checks whether two numbers are equal                                      | <code>[[ \$a -eq \$b ]]</code>                     |
| <code>-gt</code>        | Checks whether one number is greater than another                         | <code>[[ \$a -gt \$b ]]</code>                     |
| <code>==</code>         | Checks whether two strings are equal                                      | <code>[[ "\$name" == "Asif" ]]</code>              |
| <code>!=</code>         | Checks whether two strings are different                                  | <code>[[ "\$name" != "Asif" ]]</code>              |
| <code>&amp;&amp;</code> | Combines conditions using AND                                             | <code>[[ \$a -gt 5 &amp;&amp; \$b -lt 10 ]]</code> |
| <code>  </code>         | Combines conditions using OR                                              | <code>[[ \$a -gt 5    \$b -lt 10 ]]</code>         |

| Feature                                                            | <code>[ ]</code>         | <code>[[ ]]</code>          |
|--------------------------------------------------------------------|--------------------------|-----------------------------|
| Type                                                               | Traditional test         | Bash conditional expression |
| Used for conditions                                                | Yes                      | Yes                         |
| String comparison                                                  | Yes                      | Yes                         |
| File tests                                                         | Yes                      | Yes                         |
| Logical <code>&amp;&amp;</code> / <code>  </code> inside condition | Limited/different syntax | Supported directly          |
| Recommended for Bash scripts                                       | Okay                     | <b>Preferred</b>            |

Common Test Operators:

### Real DevOps Use Case Example

| Symbol / Keyword | Purpose                                       | Example                 |
|------------------|-----------------------------------------------|-------------------------|
| -eq              | Checks if two numbers are equal               | [[ \$a -eq \$b ]]       |
| -ne              | Checks if two numbers are not equal           | [[ \$a -ne \$b ]]       |
| -gt              | Checks if one number is greater than another  | [[ \$a -gt \$b ]]       |
| -lt              | Checks if one number is less than another     | [[ \$a -lt \$b ]]       |
| -ge              | Checks if one number is greater than or equal | [[ \$a -ge \$b ]]       |
| -le              | Checks if one number is less than or equal    | [[ \$a -le \$b ]]       |
| ==               | Checks if two strings are equal               | [[ "\$a" ==<br>"\$b" ]] |
| !=               | Checks if two strings are different           | [[ "\$a" !=<br>"\$b" ]] |
| -z               | Checks if a string is empty                   | [[ -z "\$name" ]]       |
| -n               | Checks if a string is not empty               | [[ -n "\$name" ]]       |
| -f               | Checks if a regular file exists               | [[ -f "\$file" ]]       |
| -d               | Checks if a directory exists                  | [[ -d "\$dir" ]]        |
| -e               | Checks if a file or directory exists          | [[ -e "\$path" ]]       |
| -r               | Checks if a file is readable                  | [[ -r "\$file" ]]       |
| -w               | Checks if a file is writable                  | [[ -w "\$file" ]]       |
| -x               | Checks if a file is executable                | [[ -x "\$file" ]]       |

### Introduction to Loops:

A loop is used to repeatedly execute a set of commands until a specific condition is met or for a specified number of times.

In simple words:

**Instead of writing the same command again and again, we use a loop to repeat it automatically.**

| Loop / Keyword | Purpose                                         | Example                                  |
|----------------|-------------------------------------------------|------------------------------------------|
| for            | Repeats commands for each item in a list        | for server in server1<br>server2 server3 |
| while          | Repeats commands while a condition is true      | while [[ \$count -lt 5 ]]                |
| until          | Repeats commands until a condition becomes true | until [[ \$count -eq 5 ]]                |

|          |                                                             |                    |
|----------|-------------------------------------------------------------|--------------------|
| do       | Starts the commands that should be repeated                 | do echo "\$server" |
| done     | Ends the loop                                               | done               |
| break    | Immediately stops the loop                                  | break              |
| continue | Skips the current iteration and continues with the next one | continue           |

## Easy Memory

for → Choose items to repeat  
 while → Repeat while condition is TRUE  
 until → Repeat until condition is TRUE  
 do → Start repeated commands  
 done → End the loop  
 break → Stop the loop  
 continue → Skip current iteration

for Loop:

A for loop is used to repeat a set of commands for each item in a list or for a specific range of values.

| Symbol / Keyword | Purpose                                       | Example                       |
|------------------|-----------------------------------------------|-------------------------------|
| for              | Starts the for loop                           | for name in Asif<br>Ravi John |
| in               | Specifies the list of items to process        | for name in Asif<br>Ravi John |
| Variable         | Stores the current item during each iteration | name                          |
| do               | Starts the commands to be repeated            | do echo "\$name"              |
| done             | Ends the for loop                             | done                          |
| \$variable       | Gets the current item's value                 | echo "\$name"                 |

## Simple Example

```
#!/bin/bash
```

```
for name in Asif Ravi John
```

```
do
    echo "Hello $name"
done
```

## Output

```
Hello Asif
Hello Ravi
Hello John
```

## Easy Memory

```
for    → Start the loop
in     → List of items
do     → Start repeated commands
$var   → Current item
done   → End the loop
```

while Loop:

**A while loop repeatedly executes a set of commands as long as a specified condition is true.**

In simple words:

**Condition is TRUE → keep repeating.**

**Condition becomes FALSE → stop.**

| Symbol / Keyword | Purpose                                             | Example                   |
|------------------|-----------------------------------------------------|---------------------------|
| while            | Starts the loop and checks the condition            | while [[ \$count -le 5 ]] |
| [[ ]]            | Tests whether the condition is true or false        | [[ \$count -le 5 ]]       |
| -le              | Checks if a number is less than or equal to another | [[ \$count -le 5 ]]       |
| do               | Starts the commands to be repeated                  | do echo "\$count"         |
| done             | Ends the while loop                                 | done                      |
| (( ))            | Performs arithmetic operations                      | ((count++))               |

## Simple Example

```
#!/bin/bash

count=1

while [[ $count -le 5 ]]
do
    echo "Count: $count"
    ((count++))
done
```

## Output

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

## Easy Memory

```
while → Check condition
do    → Start repeated commands
done  → End loop
```

```
TRUE  → Continue
FALSE → Stop
```

until Loop:

**An until loop repeatedly executes commands until a specified condition becomes true.**

In simple words:

**Condition is FALSE → keep repeating.**

**Condition becomes TRUE → stop.**

| Symbol / Keyword | Purpose                                  | Example                   |
|------------------|------------------------------------------|---------------------------|
| until            | Starts the loop and checks the condition | until [[ \$count -eq 5 ]] |
| [[ ]]            | Tests the condition                      | [[ \$count -eq 5 ]]       |
| -eq              | Checks whether two numbers are equal     | [[ \$count -eq 5 ]]       |
| do               | Starts the commands to be repeated       | do echo "\$count"         |
| done             | Ends the until loop                      | done                      |
| (( ))            | Performs arithmetic operations           | ((count++))               |

## Simple Example

```
#!/bin/bash

count=1

until [[ $count -eq 5 ]]
do
    echo "Count: $count"
    ((count++))
done
```

## Output

```
Count: 1
Count: 2
Count: 3
Count: 4
```

## Easy Memory

```
while → TRUE → Continue
until → FALSE → Continue
```

Break and Continue in Loops:

**break and continue are used to control the execution of loops.**

- **break** → completely **stops the loop**.
- **continue** → **skips the current iteration** and moves to the next iteration.

| Symbol / Keyword | Purpose                                                     | Example                                      |
|------------------|-------------------------------------------------------------|----------------------------------------------|
| break            | Immediately stops the loop                                  | if [[ \$count -eq 5 ]];<br>then break; fi    |
| continue         | Skips the current iteration and continues with the next one | if [[ \$count -eq 3 ]];<br>then continue; fi |
| if               | Checks the condition before using break or continue         | if [[ \$count -eq 5 ]]                       |
| [[ ]]            | Tests the condition                                         | [[ \$count -eq 5 ]]                          |

Nested Loops:

**A nested loop is a loop placed inside another loop.** The outer loop runs once, and for each outer-loop iteration, the inner loop runs completely.

In simple words:

**Loop inside another loop = Nested Loop**

| Symbol / Keyword | Purpose                           | Example                          |
|------------------|-----------------------------------|----------------------------------|
| for              | Starts a loop                     | for server in server1<br>server2 |
| for inside for   | Creates a nested loop             | for service in nginx ssh         |
| in               | Provides the list of items        | for service in nginx ssh         |
| do               | Starts the commands to execute    | do echo "\$service"              |
| done             | Ends the loop                     | done                             |
| \$server         | Gets the current outer-loop value | echo "\$server"                  |
| \$service        | Gets the current inner-loop value | echo "\$service"                 |

## Simple Example

```
#!/bin/bash
```

```
for number in 1 2 3
do
    for letter in A B
    do
        echo "Number: $number Letter: $letter"
```

done  
done

## Output

Number: 1 Letter: A  
Number: 1 Letter: B  
Number: 2 Letter: A  
Number: 2 Letter: B  
Number: 3 Letter: A  
Number: 3 Letter: B

Best Practices for Loops:

| Best Practice                            | Purpose                                              | Example                                             |
|------------------------------------------|------------------------------------------------------|-----------------------------------------------------|
| Use meaningful variable names            | Makes the loop easy to understand                    | for service in nginx<br>ssh                         |
| Avoid infinite loops                     | Prevents the script from running forever             | while [[ \$count -lt<br>5 ]]                        |
| Update the loop variable                 | Ensures the loop eventually stops                    | ((count++))                                         |
| Quote variables                          | Prevents problems with spaces and special characters | echo "\$filename"                                   |
| Use <code>break</code> when necessary    | Stops the loop when a required condition is met      | if [[ \$error ==<br>"yes" ]]; then break;<br>fi     |
| Use <code>continue</code> when necessary | Skips unwanted items                                 | if [[ "\$file" ==<br>*.log ]]; then<br>continue; fi |
| Keep loops simple                        | Makes scripts easier to read and troubleshoot        | for file in *.txt; do<br>echo "\$file"; done        |
| Avoid unnecessary nested loops           | Reduces complexity and improves performance          | Use one loop when possible                          |
| Test conditions before processing        | Prevents errors during automation                    | [[ -f "\$file" ]] &&<br>echo "File exists"          |
| Add comments for complex loops           | Helps others understand the automation               | # Check all application<br>servers                  |

Types of Functions:

**A function is a reusable block of commands that performs a specific task.**

Instead of writing the same commands again and again, we create a function and call it whenever needed.

| Type / Keyword                | Purpose                                              | Example                                      |
|-------------------------------|------------------------------------------------------|----------------------------------------------|
| function_name()               | Defines a function using the common Bash syntax      | check_status() { echo "Running"; }           |
| function<br>function_name     | Defines a function using the function keyword        | function check_status<br>{ echo "Running"; } |
| Function with no arguments    | Performs a fixed task without receiving input        | hello() { echo "Hello Asif"; }               |
| Function with arguments       | Receives values from the caller using \$1, \$2, etc. | greet() { echo "Hello \$1"; }                |
| Function with local variables | Creates variables available only inside the function | check() { local<br>service="\$1"; }          |
| Function with return status   | Returns a success/failure exit status                | check() { return 0; }                        |

Returning Values from Functions:

**Returning a value from a function means getting a result from a function after it finishes its task.**

In Bash, there are **two common ways**:

1. **return** → returns an **exit status** (0-255)
2. **echo** → sends a **text/value result** that can be captured

| Symbol / Keyword | Purpose                                           | Example             |
|------------------|---------------------------------------------------|---------------------|
| return           | Returns an exit status from a function            | return 0            |
| 0                | Indicates success                                 | return 0            |
| Non-zero value   | Indicates failure/error                           | return 1            |
| \$?              | Gets the exit status of the last command/function | echo "\$?"          |
| echo             | Sends a value/text from the function              | echo "\$result"     |
| \$( )            | Captures the output returned by echo              | result=\$(get_name) |

Exit Codes in Functions:

**An exit code is a number returned by a Bash function or command to tell whether it completed successfully or failed.**

In Bash:

- 0 → **Success**
- 1-255 → **Failure or different error conditions**

| Symbol / Keyword   | Purpose                                                  | Example            |
|--------------------|----------------------------------------------------------|--------------------|
| return 0           | Tells that the function completed successfully           | return 0           |
| return 1           | Tells that the function failed                           | return 1           |
| return 2           | Returns another failure/error status                     | return 2           |
| \$?                | Gets the exit code of the last executed command/function | echo "\$?"         |
| if [[ \$? -eq 0 ]] | Checks whether the function was successful               | if [[ \$? -eq 0 ]] |
| exit               | Ends the entire script with an exit code                 | exit 1             |

## Simple Example

```
#!/bin/bash
```

```
check_number() {  
    if [[ $1 -gt 10 ]]  
    then  
        return 0  
    else  
        return 1  
    fi  
}
```

```
check_number 20
```

```
if [[ $? -eq 0 ]]  
then  
    echo "Number is greater than 10"  
else  
    echo "Number is not greater than 10"  
fi
```

## Output

```
Number is greater than 10
```

| Exit Code | Meaning                                                                                     |
|-----------|---------------------------------------------------------------------------------------------|
| 0         | Success  |
| 1         | Failure  |
| 2-255     | Other errors/statuses                                                                       |
| \$?       | Get the previous exit code                                                                  |

Best practices for functions:

| Best Practice                 | Purpose                            | Example         |
|-------------------------------|------------------------------------|-----------------|
| Use meaningful function names | Makes the function's purpose clear | check_service() |

|                                    |                                                          |                                      |
|------------------------------------|----------------------------------------------------------|--------------------------------------|
| Keep functions focused             | Makes each function perform one specific task            | <code>backup_files()</code>          |
| Use arguments                      | Makes functions reusable with different inputs           | <code>check_service \$1</code>       |
| Use <code>local</code> variables   | Prevents variables from affecting the rest of the script | <code>local service="\$1"</code>     |
| Quote variables                    | Prevents problems with spaces and special characters     | <code>echo "\$service"</code>        |
| Check arguments                    | Prevents errors when required input is missing           | <code>[[ -z "\$1" ]]</code>          |
| Use <code>return</code> for status | Communicates success or failure                          | <code>return 0 /<br/>return 1</code> |
| Use <code>echo</code> for output   | Returns text or information from a function              | <code>echo "\$result"</code>         |
| Avoid unnecessary global variables | Keeps functions independent and safer                    | <code>local count=0</code>           |
| Add comments for complex functions | Helps others understand the function                     | <code># Check Nginx status</code>    |

| Best Practice            | Used In Example                      | Why?                                    |
|--------------------------|--------------------------------------|-----------------------------------------|
| Meaningful function name | <code>check_service()</code>         | Clearly explains what the function does |
| Function argument        | <code>\$1</code>                     | Allows checking different services      |
| Local variable           | <code>local service="\$1"</code>     | Keeps the variable inside the function  |
| Quoting                  | <code>"\$service"</code>             | Makes variable handling safer           |
| Condition                | <code>if [[ ... ]]</code>            | Checks service status                   |
| Exit code                | <code>return 0 /<br/>return 1</code> | Communicates success or failure         |

